

Rafael Antunes Nº55336

From Raw Detections to Real Intelligence



[Link repositório](#)

Contextualização

Este trabalho aborda o problema da transformação de dados brutos de detecção em informação inteligente para apoio à gestão de lojas de retalho. A partir de eventos gerados por sistemas de visão computacional, é necessário reconstruir trajetórias de clientes de forma automática e, posteriormente, analisar esses percursos para produzir métricas, insights e relatórios em linguagem natural com recurso a modelos de linguagem (LLMs). O objetivo principal é desenvolver um pipeline capaz de converter grandes volumes de dados anónimos em informação útil e acionável para apoio à tomada de decisão.

Dificuldades Identificadas no Dataset

A reconstrução das trajetórias torna-se complexa devido à existência de erros nos atributos demográficos, à presença simultânea de várias pessoas na loja e à ausência de deteções entre zonas. Assim, o algoritmo tem de lidar com informação parcialmente imprecisa e associar corretamente eventos intercalados, utilizando critérios temporais e probabilísticos para reconstruir percursos plausíveis dos clientes.

Algoritmo de Stitching

Dado que o dataset que foi fornecido era uma stream de eventos sem identidade, o objetivo era reconstruir quais eventos pertencem à mesma pessoa. Para isto teve-se de construir um stitcher, com uma heurística de associação, algoritmos personalizados e entre outras coisas.

Com o dataset dos eventos, também é fornecido um json com o mapa da loja, e é por aí que começa o meu stitcher, que a partir disso cria metade de uma matriz com o tempo que normalmente uma pessoa demoraria entre uma zona e outra, este calculado com o algoritmo de Dijkstra, tornando assim depois o tempo de calculo da distância ao iterar pelos eventos de **O(1)** que é o de acesso a um dicionário.

	Z_E1	Z_E2	Z_C1	Z_C2	Z_C3	Z_CK	Z_N1	Z_N2	Z_N3	Z_N4	Z_N5	Z_N6	Z_N7	Z_N8	Z_N9	Z_N10	Z_S1	Z_S2	Z_S3	Z_S4	Z_S5	Z_S6	Z_S7
Z_E1	0																						
Z_E2	22	0																					
Z_C1	8	18	0																				
Z_C2	14	8	10	0																			
Z_C3	24	14	20	10	0																		
Z_CK	28	26	20	18	20	0																	
Z_N1	20	33	15	25	35	35	0																
Z_N2	29	20	25	15	25	33	20	0															
Z_N3	39	29	35	25	15	35	40	20	0														
Z_N4	38	51	33	43	53	53	18	36	54	0													
Z_N5	47	38	43	33	43	51	36	18	36	18	0												
Z_N6	57	47	53	43	33	53	54	36	18	36	18	0											
Z_N7	56	69	51	61	71	71	36	54	72	18	36	54	0										
Z_N8	65	56	61	51	61	69	54	36	54	36	18	36	18	0									
Z_N9	75	65	71	61	51	71	72	54	36	54	36	18	36	18	0								
Z_N10	73	73	68	68	86	53	53	35	35	35	20	18	20	0									
Z_S1	45	58	40	50	60	60	25	45	65	20	38	56	38	56	74	55	0						
Z_S2	54	45	50	40	50	58	45	25	45	61	43	61	79	61	79	78	70	0					
Z_S3	64	54	60	50	40	60	65	45	25	79	61	43	97	79	61	78	90	70	0				
Z_S4	58	71	53	63	73	73	38	56	68	20	38	50	20	33	35	15	40	81	93	0			
Z_S5	67	58	63	53	63	71	56	38	56	38	20	38	35	33	35	15	58	63	81	30	0		
Z_S6	77	67	73	63	53	73	68	56	38	50	38	20	35	33	35	15	70	81	63	30	30	0	
Z_S7	88	85	83	81	71	91	68	68	56	50	50	38	35	33	20	15	70	93	81	30	30	30	0

Visualização da Matriz de Tempo

Ao iterar pelos eventos, vai-se introduzindo pessoas novas cada vez que alguém entra nas zonas de entrada, removendo-as depois quando saem nestas mesmas zonas, na zona de checkout geral ou quando se perdem por mais de 5 minutos.

Para se associar os eventos à pessoa ideal é usada a heurística desenvolvida, cuja dá um valor de 0 a 100 do quão é provável ser daquele evento, para este valor ser alto ele precisa de pontuar alto em três categorias, **diferença do tempo** de chegada com a estimada (40 Pontos), **similaridade no género** (30 Pontos) e **similaridade na faixa etária** (30 Pontos).

No final, a complexidade temporal deste algoritmo totalizou-se em $O(N.P)$, N sendo a quantidade de eventos e P sendo a quantidade de Pessoas ativas.

Pipeline Analítico

Para as métricas calculadas foram calculadas todas as indicadas no enunciado, estas sendo Métricas de tráfego, Métricas por zona, Funil de cliente, Segmentação demográfica e Detecção de anomalias.

[Exemplo do Metrics.json](#)

Engenharia de Prompts

Arquitetura do Sistema de Prompts

O módulo insights.py segue uma abordagem por categoria: para cada uma das cinco categorias (tráfego, zonas, funil, anomalias, demográfico), é feita uma chamada separada ao modelo com apenas as métricas relevantes para essa categoria. Esta decisão foi tomada por duas razões:

1. **Foco:** um modelo de 3B parâmetros tem dificuldade em manter coerência sobre um JSON de 800 linhas. Filtrar as métricas por categoria reduz o contexto e melhora a qualidade.
2. **Controlabilidade:** é possível pedir um número exacto de insights por categoria e verificar separadamente a qualidade de cada um.

Cada categoria tem um filtro Python (`_filtro_trafego`, `_filtro_zona`, etc.) que extrai apenas as chaves relevantes do metrics.json antes de as enviar ao modelo.

Componentes dos Prompts

Cada prompt é composto por quatro blocos:

1. **instrucoes.txt** — define o papel do modelo ("especialista em comportamento de consumidores"), o que cada insight deve conter, e a escala de confiança (1.0 = dado directo, 0.7 = inferência, 0.5 = hipótese).
2. **regras.txt** — regras obrigatórias: usar apenas os dados fornecidos, números concretos obrigatórios, responder apenas com JSON válido.
3. **Exemplos few-shot** (apenas estratégia B) — ficheiro por categoria com exemplos de bons e maus insights, seguindo a tabela do enunciado.

4. **Pedido + schema** — instrução de quantos insights gerar, por que temas, e o schema JSON esperado.

Estratégia A - Zero-Shot

A estratégia A envia directamente as instruções, as regras, as métricas filtradas e o schema de output, sem exemplos. O modelo recebe apenas:

{instrucoes} + {regras} + CATEGORIA + MÉTRICAS + pedido + schema

Estratégia B — Few-Shot

A estratégia B insere um bloco de exemplos entre as regras e as métricas reais. Os exemplos mostram explicitamente um insight mau (vago, sem números) e um insight bom (específico, com valores concretos e recomendação executável) para cada categoria. O prompt termina com o reforço: *"cada insight deve ter números concretos das métricas. ZERO afirmações vagas."*

Comparação Quantitativa A vs. B

A função avaliar_insights foi reestruturada para adotar o padrão da indústria de *LLM-as-a-Judge*. Em vez de depender de heurísticas rígidas e procurar correspondências exatas de texto, o sistema utiliza o próprio modelo (llama3.2:3b) no papel de "Gestor de Retalho Sênior" para ler e avaliar criticamente cada insight gerado pelas estratégias.

O modelo avaliador atribui uma nota de 0.0 a 1.0 baseando-se em três pilares semânticos e de negócio:

Critério	Método de Avaliação (Prompt do Avaliador)
Precisão Matemática	O LLM verifica se a observação faz sentido face às métricas originais, penalizando severamente números alucinados ou absurdos.
Acionabilidade da Recomendação	Avalia se a recomendação sugere ações físicas, táticas e específicas na loja, em oposição a jargão corporativo vago e inútil.
Derivação de Inteligência	Premeia insights que deduzem cálculos úteis (como percentagens ou diferenças reais) em vez de apenas copiarem as chaves do ficheiro JSON de origem.

Número de Insights por Categoria

O número de insights por categoria foi fixado explicitamente:

Categoria	N insights	Justificação
tráfego	3	Um por tema: afluência global, hora de pico, variação diária
zonas	2	Top zona e zona problemática
funil	3	Conversão, perda de tráfego, perfil de não-compradores
anomalias	dinâmico	Um por grupo de anomalias detectadas (anomaly_groups)

Avaliação

Harness de Avaliação

O harness evaluate.py é executável com:

```
python evaluate.py --data events_validation.csv --output evaluation_report.json
```

Recebe um CSV no mesmo formato do dataset de treino (com anomalias injectadas conhecidas) e corre o pipeline completo: stitching → analytics → insights → verificação. As métricas calculadas são:

- **Consistência:** % de trajetórias onde nenhum par de visitas se sobrepõe temporalmente. Qualquer valor inferior a 100% indica um bug no stitcher.
- **Cobertura:** % de eventos do CSV original que foram associados a alguma trajetória.
- **Compleitude:** % de trajetórias que começam em Z_E e terminam em Z_E ou Z_CK.
- **Deteção de anomalias:** % das anomalias injectadas corretamente identificadas nos insights.

- **Precisão numérica:** % de valores numéricos nos insights verificáveis no `metrics.json`.
- **Ausência de alucinação:** % de afirmações factuais no relatório verificáveis no `metrics.json`.

Limitações Conhecidas do Sistema

- **Stitcher — sensibilidade a grupos homogêneos.**
Quando múltiplas pessoas com os mesmos atributos demográficos transitam entre zonas com tempos semelhantes, o algoritmo pode trocar identidades. Este é um caso limite do problema de *data association* e não é totalmente evitável com heurísticas simples.
- **Insights — alucinação numérica residual.**
O modelo `llama3.2:3b`, mesmo com `few-shot` e temperatura 0, ocasionalmente inventa percentagens que não constam do `metrics.json`. A métrica de verificação no harness quantifica esta taxa, mas não a elimina.
- **Insights e Comparação — tempo de execução.**
Mesmo com o modelo `llama3.2:3b`, o tempo de execução dos insights e da comparação é demasiado extenso, os insights demorando por volta de 18 minutos e a comparação a nem sequer acabando.

Conclusão

O que funcionou

O stitcher foi a parte do trabalho que funcionou melhor, e com uma forma criativa e eficiente de recriar os caminhos de cada pessoa, uma heurística presumivelmente admissível e resultados bons e credíveis.

A abordagem de prompting por categoria (em vez de um único prompt com todas as métricas) melhorou significativamente a qualidade dos insights. Com um modelo de 3B parâmetros, reduzir o contexto por chamada é mais eficaz do que aumentar o contexto por chamada.

O que não funcionou

Os modelos demoram demasiado tempo a acabar a resposta, isto pode ser pela extensão do prompt, pela quantidade de informação ou pelas especificações do computador.

O que faria de diferente

Teria tentado tirar mais peso de cima dos LLM's com processamento direto to computador, ou código mais eficiente ao longo do trabalho, tendo mais atenção a como fazia as coisas, pois o trabalho acabou por ser feito à pressa.

Em vez de usar heurísticas ou partes mais “hardcoded” neste código, ter tido uma aproximação por machine learning a isto, ter investigado como poderia ter usado métodos de aprendizagem não supervisionada.